PRÁCTICA 3 - PLN

Trabajamos con el fichero `largo.bin`; desde la terminal de Linux, usando el comando `xxd`, podemos ver su representación en hexadecimal:

```
usuario_local@pto1103:~/Downloads$ xxd largo.bin
00000000: 1835 210b 2821 0b1f 281a 1425 0b17 180b  .5!.(!..(..%....
00000010: 1f14 0b20 3514 2116 1b14 470b 1718 0b16  ... 5.!...G.....
00000020: 282c 220b 2122 2015 2518 0b21 220b 2428  (,".!" .%..!".$(
00000030: 1c18 2522 0b14 1622 2517 1425 2018 470b  ..%"..."%..% .G.
00000040: 2122 0b1b 140b 2028 161b 220a 271c 1820  !".... (.."."...
00000050: 2322 0b24 2818 0b29 1c29 1c32 140b 2821  #".$(..).).2..(!
00000060: 0b1b 1c17 141f 1a22 0b17 180b 1f22 260b  ......."....."&.
00000070: 1718 0b1f 1421 2d14 0b18 210b 1426 271c  .....!-...!..&'.
00000080: 1f1f 1825 2247 0b14 1714 251a 140b 1421  ...%"G....%....!
00000090: 271c 1a28 1447 0b25 2216 1c32 210a 191f  '..(.G.%"..2!...
000000a0: 1416 220b 2c0b 1a14 1f1a 220b 1622 2525  ..".,....."..%%
000000b0: 1817 2225 460b 2835 2114 0b22 1f1f 140b  .."%F.(5!.."....
000000c0: 1718 0b14 1f1a 220b 2014 3226 0b29 1416  ......". .2&.)..
000000d0: 140b 2428 180b 1614 2521 1825 2247 0b26  ..$(....%!.%"G.&
000000e0: 141f 231c 1622 3221 0b1f 1426 0b20 1432  ..#.."2!...&. .2
000000f0: 260a 2122 161b 1826 470b 1728 181f 2226  &.!"...&G..(.."&
00000100: 0b2c 0b24 2818 1525 1421 2722 260b 1f22  .,.$(..%.!'"&.."
00000110: 260b 2614 3215 1417 2226 470b 1f14 2127  &.&.2..."&G...!'
00000120: 181d 1426 0b1f 2226 0b29 1c18 2521 1826  ...&.."&.)..%!.&
00000130: 470b 141f 1a28 3221 0b23 141f 2220 1c21  G....(2!.#.." .!
```

También podríamos hacer `xxd -b largo.bin` para verlo en formato binario.

Nota: hay varios archivos `.bin` subidos al Campus. Puedes coger cualquiera para la tarea, porque si descifras uno, descifras todos (solo cambia principalmente la longitud del texto).

Estando en hexadecimal, vamos a tratar de descifrarlo. Para ello, usamos Python (trabajando con bytes, que esto es tomando los caracteres de la imagen anterior de 2 en 2, porque 1 byte = 2 bits de estos).

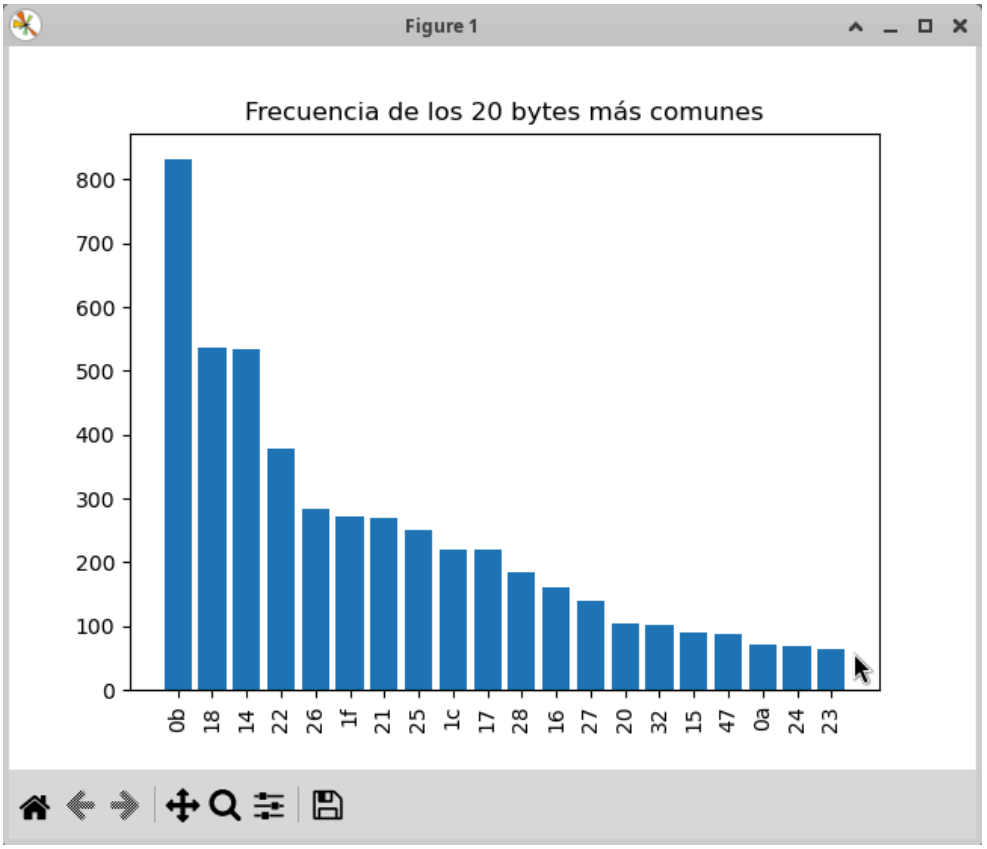
Python

```python
from collections import Counter
import matplotlib.pyplot as plt

# 1. LEER EL ARCHIVO BINARIO
with open("largo.bin", "rb") as f:
    datos = f.read()

print(f"Tamaño total del archivo: {len(datos)} bytes")

# 2. CONTAR FRECUENCIA DE CADA BYTE
frecuencias = Counter(datos)

print(f"Número de valores distintos de byte: {len(frecuencias)}")

print("\nLos 10 bytes más frecuentes:\n")

for byte, freq in frecuencias.most_common(10):
    print(f"{byte:02x}  ->  {freq}")
```

```python
# 3. BYTE MÁS FRECUENTE (posible espacio)
byte_mas_comun = frecuencias.most_common(1)[0][0]

print("\nByte más común:", f"{byte_mas_comun:02x}")

# 4. Prueba: asumir que el más frecuente es espacio (0x20)
texto_prueba = datos.replace(bytes([byte_mas_comun]), b' ')

print("\nPrimeros 500 bytes con posible espacio reemplazado:\n")
print(texto_prueba[:500])

# 5. VISUALIZACIÓN DE LOS 20 MÁS FRECUENTES
top = frecuencias.most_common(20)

bytes_vals = [f"{b:02x}" for b, _ in top]
freqs = [f for _, f in top]

plt.bar(bytes_vals, freqs)
plt.title("Frecuencia de los 20 bytes más comunes")
plt.xticks(rotation=90)
plt.show()
```

En primer lugar, hacemos un análisis de frecuencia, con lo que obtenemos:

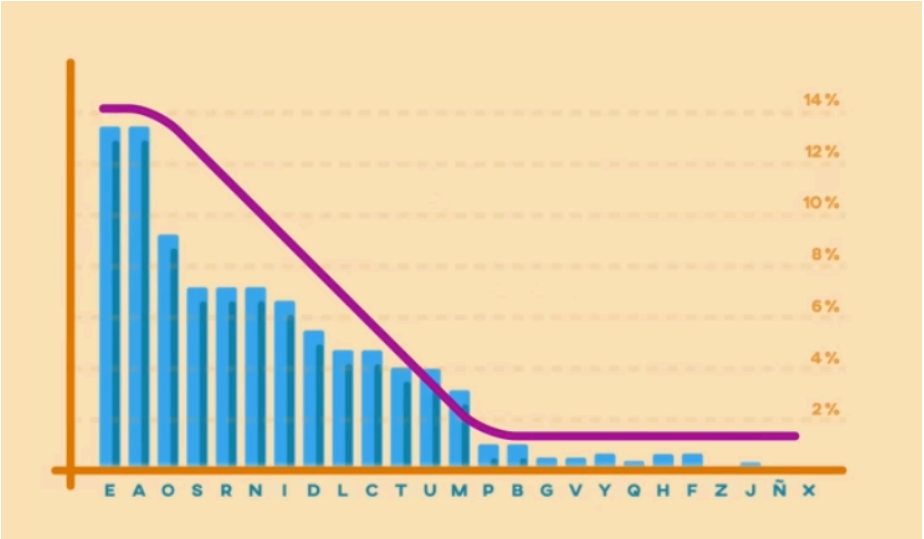
```
Tamaño total del archivo: 5225 bytes
Número de valores distintos de byte: 36

Los 10 bytes más frecuentes:

0b  ->  831
18  ->  537
14  ->  533
22  ->  379
26  ->  283
1f  ->  271
21  ->  270
25  ->  251
1c  ->  221
17  ->  220

Byte más común: 0b
```



Con esto y un pequeño análisis exploratorio de Tom, hemos llegado a las siguientes conclusiones:

- 0b es el espacio
- 18 es la letra E
- 14 es la letra A

Eso así a simple vista, únicamente atendiendo a la frecuencia y comparándola con la frecuencia de las letras más usadas en españo:



El punto más débil de la criptografía no es la cuántica

Ahora, Tom logró descifrar el código hexadecimal pasándolo a legible y se obtiene el siguiente resultado:

```
e[53]n un lugar de la m[53]ancha[71] de cuyo nombre no quiero
acordarme[71] no ha muchoWtiempo que vivi[73]a ……
```

Hicimos un análisis de estos números entre corchetes que aparecen en mitad del texto y descubrimos que:

`[53]` = Mayúscula la letra anterior

`[71]` = la coma (,)

`W` = el salto de línea (Enter)

`[73]` = la tilde (') en la letra anterior

… y hay más de estos números <u>decimales</u> (si se pasa a hexadecimal, el 53 es un 35, como se ve en la siguiente figura).

Además, vimos que cada uno de estos coincidía con un byte de la cadena hexadecimal. Sustituyendo cada uno de los caracteres de la frase obtenida con cada uno de los bytes (conjunto de 2 caracteres) del código hexadecimal:

```
00000000: 1835 210b 2821 0b1f 281a 1425 0b17 180b  .5!.(!..(..%....
00000010: 1f14 0b20 3514 2116 1b14 470b 1718 0b16  ... 5.!...G.....
```

El 18 = E, 35 = letra mayúscula [53], 21 = N, 0b = espacio, etc. etc. etc.

¡¡ TODO CUADRA !!

Números apuntados por el profesor en la pizarra que tenemos que averiguar su significado:

65 - 20 = 45

(+32)

97 - 20 = 77

- ¿Qué es el 65? La codificación de la A mayúscula. Es decir, es el nº o el byte que si yo interpreto una cadena de bytes como ASCII, interpreto como si fuera una A.
- ¿Y el 97? La a minúscula
- ¿Y el 32? Pues entonces, si a la A mayúscula le sumo 32, tengo la a minúscula.
- ¿Entonces 45 y 77?

¿Qué es 77? El desplazamiento para descifrar las letras codificadas como código César (la A tendría que ser el 1, pero para complicarlo más, la A será el 1+77=78; y así con la B = 2+77; etc. ** corregir

<u>La pregunta importante</u>: ¿Qué es el 20? (40, 50 y 100 también son importantes) es un número arbitrario que supone que es la a, entonces el 21 será la b, etc.

ENTONCES La primera parte de la tarea era, dado un texto raro cifrado, pasarlo a texto legible en español. La segunda parte de la tarea será, dado un texto legible en español, pasarlo a texto raro cifrado.

La entrega que querrá el profe es un script que sea capaz de pasar de un tipo de texto a otro (ya sea se lo pasamos en texto raro y que el código lo pase a legible, o viceversa, se lo pasamos en legible y que nuestro script lo pase a texto raro).